

PART-I MCQ

- | | | |
|-------|-------|-------|
| 1. B | 11. B | 21. B |
| 2. B | 12. B | 22. A |
| 3. B | 13. B | 23. A |
| 4. B | 14. B | 24. B |
| 5. B | 15. A | 25. B |
| 6. C | 16. B | |
| 7. C | 17. A | |
| 8. C | 18. B | |
| 9. B | 19. C | |
| 10. A | 20. B | |

PART-II : Open Questions:-

a) Deriving Backpropagation Algorithm:-

i) Network:- 3 Layer Feed Forward Neural Network with logistic Sigmoid Activation Functions:-

ii) Cross Entropy Equation:-

$$J_{ce}(W) = - \sum_{p=1}^n [t_p \ln z_p + (1-t_p) \ln (1-z_p)]$$

where t_p is target output for input $\bar{x}_p$,
 z_p is network output for $\bar{x}_p$

iii) Forward Pass Calculations:-

1. Activations of Hidden Layers computations:-

$$h_j = \sigma \left(\sum_i w_{ji}^{(1)} x_i \right), \quad \sigma(x) = \frac{1}{1+e^{-x}}$$

2. Network Outputs Calculations

$$z_p = \sigma \left(\sum_k w_{kj}^{(2)} h_j \right)$$

iv) Gradient Calculations for Backward Propagation:

1. error at the output layer: $\delta_k = z_p - t_p$
because the crossentropy loss derivative wrt z_p is

$$\frac{\partial J_{ce}}{\partial z_p} = \frac{z_p - t_p}{z_p(1-z_p)}$$

$\therefore z_p = \sigma(a_k)$, $\sigma'(a_k) = z_p(1-z_p)$, thus
error simplifies to be $\delta_k = z_p - t_p$

v) computing error at hidden layer:

$$\delta_j = h_j(1-h_j) \sum_k w_{kj}^{(2)} \delta_k$$

This concludes the backpropagation from the output to hidden layer. Next is the weight updates as per error calculated

vi) Weight Updates:

1. First we update the weights at b/w Output and hidden Layers

$$w_{kj}^{(2)} \leftarrow w_{kj}^{(2)} - \eta \delta_k h_j$$

where η is learning rate.

2. Next, we update weights between Input & hidden Layers.

$$w_{ji}^{(1)} \leftarrow w_{ji}^{(1)} - \eta \delta_j x_i$$

3. Batch gradient Descent Update Rule:

For batch gradient descent, the update of weights is accumulated over all training examples before updating the weights.

$$w_{kj}^{(2)} \leftarrow w_{kj}^{(2)} - \eta \sum_p \delta_k^{(p)} h_j^{(p)}$$

$$w_{ji}^{(1)} \leftarrow w_{ji}^{(1)} - \eta \sum_p \delta_j^{(p)} x_i^{(p)}$$

b) CrossEntropy Loss over MSE while using Sigmoid outputs in classification:—

i) Gradient Behavior :-

CrossEntropy Loss gradient $\propto$ error ($z_p - t_p$) which is large when error is large leading to faster updates. But in case of MSE loss gradient is very small when output z_p is close to 0 or 1. $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, thus $\sigma(x)$ becomes small leading to vanishing gradient meaning slow convergence.

ii) Interpretation of Output:-

The CEL is derived from likelihood of correct class under Bernoulli's distribution, which is a Natural choice for Binary Classifications, as it measures the difference b/w predicted probability and true distribution. On the other hand MSE is more suited for regression, where goal is to minimize the sum of squared errors (diff. b/w predicted and actual). So it will not align with probabilistic interpretation of classification outputs.

iii) Avoiding Saturation:-

Sigmoid produces very low gradients when input is very large or very small. CEL avoids this bcz its gradient doesn't depend on sigmoid function unlike MSE.

CSCE 5210: FUNDAMENTALS OF AI
HOMEWORK 02**PART 03: CODING LAB -PDF REPORT****Implementation of a deep Multi-Layer Perceptron (MLP) for the MNIST dataset**

1. First, I have imported the required Pytorch libraries, then downloaded the dataset, checked for GPU, if available run the network on it else on CPU.
2. Then normalized the dataset, then split the 60,000 training images into 50,000 training and 10,000 validation datasets. It also has 10,000 testing images, and the loaders for all three of them is generated.
3. Then defined the DeepMLP class with 3 hidden layers with 256,128,64 neurons respectively and 1 output layer with 10 neurons corresponding to the digits or target classes of MNIST data. The Input will have $28 \times 28 = 784$ neurons or flattened images.
4. Then define the evaluation of model, and prep for training the model using defined evaluate function under given parameters, epochs =5, batch size =64, Adam optimization(lr=0.001), cross entropy loss function.
5. After completing the training, the test data is passed to evaluate function to get the final loss, and accuracy of the network.
6. After training the final Validation Loss: 0.0743, and the final Validation Accuracy: 97.60%.
7. The Testing loss came to be 0.0739 and Test Accuracy is 97.80%.
8. The Test loss, accuracy of the tutorial 2Layer MLP in the ppt is 0.0760, 97.66% respectively, Although there is not much difference, there is a slight increase in the deeper MLP.
9. **Discussion Question:** Comparison of this DeepMLP with 2layer MLP in Tutorial.
 - >The test accuracy of the deeper model will be higher than the baseline 2layer one.
 - >But the deeper layer will have more training time than the simple 2layer, the dropout Regularization in deeper layer will make sure that network is not overfitted, which is not a concern in the 2 layered one.
 - > The feature extraction is easier in DeepMLP as the due to increased complexity than the 2 layered MLP.